

Hive

wget <https://apache.root.lu/hive/hive-3.1.3/apache-hive-3.1.3-bin.tar.gz>

Complete Working Hive Practical Steps (Final)

Application using Apache Hive for Flight Information System.

1. Start Hadoop

Start HDFS:

```
start-dfs.sh
```

Start YARN:

```
start-yarn.sh
```

Verify:

```
jps
```

Expected:

```
NameNode  
DataNode  
SecondaryNameNode  
ResourceManager  
NodeManager
```

2. Download Hive

Go to home:

```
cd ~
```

Download Hive:

```
wget https://apache.root.lu/hive/hive-3.1.3/apache-hive-3.1.3-bin.tar.gz
```

Extract:

```
tar -xvzf apache-hive-3.1.3-bin.tar.gz
```

Rename:

```
mv apache-hive-3.1.3-bin hive
```

3. Configure Environment Variables

Open `.bashrc`

```
nano ~/.bashrc
```

Add these lines at end:

```
export HIVE_HOME=~/.hive
export PATH=$PATH:$HIVE_HOME/bin
export HADOOP_HOME=/usr/local/hadoop
```

Reload:

```
source ~/.bashrc
```

Verify:

```
hive --version
```

Expected:

4. Create Hive Directories in HDFS

If NameNode is in safe mode:

```
hdfs dfsadmin -safemode leave
```

Create directories:

```
hdfs dfs -mkdir /tmp
```

```
hdfs dfs -chmod g+w /tmp
```

```
hdfs dfs -mkdir -p /user/hive/warehouse
```

```
hdfs dfs -chmod g+w /user/hive/warehouse
```

5. Configure Hive

Go to config folder:

```
cd ~/hive/conf
```

Create hive-env:

```
cp hive-env.sh.template hive-env.sh
```

Edit:

```
nano hive-env.sh
```

Add:

```
export HADOOP_HOME=/usr/local/hadoop
```

6. Create Clean hive-site.xml

```
nano hive-site.xml
```

Paste EXACTLY:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>

<property>
  <name>javax.jdo.option.ConnectionURL</name>
  <value>jdbc:derby:/home/frozn/hive/metastore_db;create=true</value>
</property>

<property>
  <name>javax.jdo.option.ConnectionDriverName</name>
  <value>org.apache.derby.jdbc.EmbeddedDriver</value>
</property>

<property>
  <name>hive.metastore.warehouse.dir</name>
  <value>/user/hive/warehouse</value>
</property>

</configuration>
```

Save and exit.

7. Remove Old Metastore Errors

```
rm -rf ~/metastore_db
```

```
rm -rf ~/hive/metastore_db
```

```
rm -f ~/derby.log
```

```
rm -f ~/hive/derby.log
```

8. Initialize Hive Schema

```
schematool -dbType derby -initSchema
```

Expected:

```
schemaTool completed
```

Ignore SLF4J warnings.

9. Start Hive

```
hive
```

10. Create Database

```
CREATE DATABASE flightdb;
```

Use database:

```
USE flightdb;
```

11. Create Flights Table

```
CREATE TABLE flights (  
    flight_id INT,  
    airline STRING,  
    source STRING,  
    destination STRING,  
    departure_delay INT,  
    flight_date STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

12. Create External Table

```
CREATE EXTERNAL TABLE flight_external (  
    flight_id INT,  
    airline STRING,  
    source STRING,  
    destination STRING,  
    departure_delay INT,  
    flight_date STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
LOCATION '/user/hive/external_flights';
```

13. Create CSV File

Exit Hive:

```
exit;
```

Create file:

```
nano flights.csv
```

Paste:

```
1,Indigo,Pune,Delhi,10,2008-01-01  
2,AirIndia,Mumbai,Bangalore,20,2008-01-01  
3,SpiceJet,Delhi,Chennai,15,2008-01-02  
4,Vistara,Pune,Hyderabad,5,2008-01-02  
5,Indigo,Mumbai,Delhi,25,2008-01-03
```

14. Upload File to HDFS

```
hdfs dfs -mkdir /flightdata
```

```
hdfs dfs -put flights.csv /flightdata
```

Verify:

```
hdfs dfs -ls /flightdata
```

15. Load Data into Hive

Start Hive again:

```
hive
```

Use database:

```
USE flightdb;
```

Load data:

```
LOAD DATA INPATH '/flightdata/flights.csv'  
INTO TABLE flights;
```

16. Display Data

```
SELECT * FROM flights;
```

17. Insert New Values

```
INSERT INTO flights VALUES  
(6, 'Akasa', 'Pune', 'Goa', 12, '2008-01-03');
```

18. Alter Table

```
ALTER TABLE flights  
ADD COLUMNS (status STRING);
```

Check structure:

```
DESCRIBE flights;
```

19. Create Another Table for Join

```
CREATE TABLE airlines (  
    airline STRING,  
    country STRING  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ',';
```

Insert values:

```
INSERT INTO airlines VALUES  
( 'Indigo', 'India' ),  
( 'AirIndia', 'India' ),
```



```
('SpiceJet','India'),  
( 'Vistara','India'),  
( 'Akasa','India');
```

20. Join Tables

```
SELECT f.flight_id,  
       f.airline,  
       a.country  
FROM flights f  
JOIN airlines a  
ON (f.airline = a.airline);
```

21. Optimization using Bucketing (Replacement for Index)

```
CREATE TABLE flight_indexed (  
    flight_id INT,  
    airline STRING,  
    source STRING,  
    destination STRING,  
    departure_delay INT,  
    flight_date STRING  
)  
CLUSTERED BY (airline) INTO 4 BUCKETS;
```

Insert data:

```
INSERT INTO flight_indexed  
SELECT flight_id,  
       airline,  
       source,  
       destination,  
       departure_delay,
```

```
        flight_date  
FROM flights;
```

Viva explanation:

Hive 3.x deprecated CREATE INDEX support.
Bucketing is used instead for optimization.

22. Average Departure Delay Per Day

```
SELECT flight_date,  
       AVG(departure_delay) AS avg_delay  
FROM flights  
GROUP BY flight_date;
```

Expected:

2008-01-01	15.0
2008-01-02	10.0
2008-01-03	18.5

23. Drop Table and Database

```
DROP TABLE airlines;
```

Check databases:

```
SHOW DATABASES;
```

If `flightdb` exists:

```
DROP DATABASE flightdb CASCADE;
```